

Лабораторная работа №3:

Разработка веб-приложения на Spring Boot.

Введение в IoC/DI

Цель работы

Научиться создавать Spring Boot приложение, освоить принципы инверсии управления (IoC) и внедрения зависимостей (DI), изучить три способа конфигурации Spring-компонентов, а также научиться разрешать конфликты при наличии нескольких бинов одного типа.

Теоретическая часть

Spring Boot — надстройка над Spring Framework, которая позволяет быстро создавать production-grade приложения с минимальной конфигурацией.

Слоистая архитектура:

- **@Controller** — принимает HTTP-запросы
- **@Service** — содержит бизнес-логику
- **@Repository** — работает с данными (в этой работе будем имитировать)

Ключевые понятия:

- **Bean** — объект, управляемый Spring IoC-контейнером
- **IoC (Inversion of Control)** — управление созданием объектов передано контейнеру
- **DI (Dependency Injection)** — внедрение готовых зависимостей в объект

Ход выполнения работы

Часть 0. Подготовка

1. Создайте проект через Spring Initializr со следующими параметрами:

- Project: Maven
- Language: Java 17+
- Dependencies: **Spring Web, Spring Boot DevTools**
- Название: **spring-lab3-notifications**

2. Создайте структуру пакетов:

```
org.example
  controller
    service
      config
```

Часть 1. Знакомство с Spring Boot Web

1. В пакете controller создайте класс HelloController:

```
1 package org.example.controller;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController
7 public class HelloController {
8
9     @GetMapping("/hello")
10    public String sayHello() {
11        return "Привет, Spring Boot!";
12    }
13 }
14
```

2. Запустите главный класс (с @SpringBootApplication).

3. Откройте браузер: <http://localhost:8080/hello>

4. Убедитесь, что видите сообщение.

Самостоятельное задание к Части 1

1. Добавьте еще один метод в HelloController, который обрабатывает GET-запрос на /goodbye и возвращает "До свидания, Spring Boot!".
2. Добавьте метод, который принимает параметр name через @RequestParam и возвращает "Привет, name!". Например, запрос /greet?name=Иван должен вернуть "Привет, Иван!".
3. (Дополнительное задание) Добавьте метод, который принимает два параметра (name и age) и возвращает сообщение с информацией о пользователе.

Часть 2. Проблема жестких связей

Задача: Создать систему уведомлений без использования Spring, увидеть её недостатки.

1. В пакете `service` создайте интерфейс:

```
1 package org.example.service;
2
3 public interface MessageService {
4     void sendMessage(String message, String recipient);
5 }
6
```

2. Создайте две реализации:

```
1 package org.example.service;
2
3 public class EmailService implements MessageService {
4     @Override
5     public void sendMessage(String message, String recipient) {
6         System.out.println("EMAIL to " + recipient + ": " +
7 message);
8     }
9 }
```

```
1 package org.example.service;
2
3 public class SmsService implements MessageService {
4     @Override
5     public void sendMessage(String message, String recipient) {
6         System.out.println("SMS to " + recipient + ": " +
7 message);
8     }
9 }
```

3. Создайте класс-менеджер с **жесткой привязкой**:

```
1 package org.example.service;
2
3 public class NotificationManager {
4     private final EmailService emailService;
5
6     public NotificationManager() {
7         this.emailService = new EmailService(); // жесткая связь
8     }
9
10    public void notify(String message, String recipient) {
11        emailService.sendMessage(message, recipient);
12    }
13 }
14
```

4. Используйте его в контроллере:

```
1 package org.example.controller;
2
3 import org.example.service.NotificationManager;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RequestParam;
6 import org.springframework.web.bind.annotation.RestController;
7
8 @RestController
9 public class NotificationController {
10
11     @GetMapping("/notify")
12     public String notify(@RequestParam String message,
13         @RequestParam String email) {
14         NotificationManager manager = new NotificationManager();
15         manager.notify(message, email);
16         return "Уведомление отправлено (жесткая связь)";
17     }
18 }
```

5. Запустите и протестируйте:

`http://localhost:8080/notify?message=Hi&email=test@test.com`

Самостоятельное задание к Части 2

1. Модифицируйте `NotificationManager` так, чтобы он мог работать с **любым** сервисом, реализующим `MessageService`, а не только с `EmailService`. Измените конструктор и поле соответствующим образом.
2. Создайте третий сервис `PushService`, который выводит сообщение о push-уведомлении. Как можно использовать его с вашим модифицированным `NotificationManager`?
3. Ответьте письменно: какие проблемы остаются в коде даже после ваших улучшений? Почему такой подход все еще неудобен для больших проектов?

Часть 3. Java Config (явная конфигурация через `@Configuration` и `@Bean`)

Задача: Перевести приложение на Spring, используя **явное объявление бинов** в конфигурационном классе.

Важно: В этой части **не используются** аннотации `@Service` и `@Component`. Все бины создаются вручную через `@Bean`.

1. Создайте конфигурационный класс в пакете `config`:

```
1 package org.example.config;
2
3 import org.example.service.EmailService;
4 import org.example.service.NotificationManager;
5 import org.example.service.SmsService;
6 import org.springframework.context.annotation.Bean;
7 import org.springframework.context.annotation.Configuration;
```

```

8
9 @Configuration
10 public class AppConfig {
11
12     @Bean
13     public EmailService emailService() {
14         return new EmailService();
15     }
16
17     @Bean
18     public SmsService smsService() {
19         return new SmsService();
20     }
21
22     @Bean
23     public NotificationManager notificationManager() {
24         // ЯВНО указываем, какой бин внедряем
25         return new NotificationManager(emailService());
26     }
27 }
28

```

2. Модифицируйте NotificationManager — теперь зависимость приходит извне:

```

1 package org.example.service;
2
3 public class NotificationManager {
4     private final MessageService messageService;
5
6     // Конструктор для внедрения зависимости
7     public NotificationManager(MessageService messageService) {
8         this.messageService = messageService;
9     }
10
11     public void notify(String message, String recipient) {
12         messageService.sendMessage(message, recipient);
13     }
14 }
15

```

3. Обновите контроллер, чтобы получать бин из контекста:

```

1 package org.example.controller;
2
3 import org.example.service.NotificationManager;
4 import org.springframework.context.ApplicationContext;
5 import org.springframework.web.bind.annotation.GetMapping;
6 import org.springframework.web.bind.annotation.RequestParam;
7 import org.springframework.web.bind.annotation.RestController;
8
9 @RestController
10 public class NotificationController {
11
12     private final ApplicationContext context;
13
14 }
15

```

```

13
14     public NotificationController(ApplicationContext context) {
15         this.context = context;
16     }
17
18     @GetMapping("/notify")
19     public String notify(@RequestParam String message,
20 @RequestParam String email) {
21         NotificationManager manager = context.getBean(
22 NotificationManager.class);
23         manager.notify(message, email);
24         return "Уведомление отправлено через Java Config";
25     }
26 }

```

4. Обновите главный класс, чтобы подключить конфигурацию:

```

1 package org.example;
2
3 import org.example.config.AppConfig;
4 import org.springframework.boot.SpringApplication;
5 import org.springframework.boot.autoconfigure.
    SpringApplication;
6 import org.springframework.context.annotation.Import;
7
8 @SpringBootApplication
9 @Import(AppConfig.class) // Подключаем нашу Java-конфигурацию
10 public class Application {
11     public static void main(String[] args) {
12         SpringApplication.run(Application.class, args);
13     }
14 }
15

```

5. Запустите и проверьте.

Результат: Spring создает бины по инструкциям в `AppConfig`. Внедряется `EmailService`, потому что мы явно вызвали `emailService()` в методе `notificationManager()`.

Самостоятельное задание к Части 3

1. Измените метод `notificationManager()` в `AppConfig` так, чтобы он внедрял `smsService()` вместо `emailService()`. Запустите и убедитесь, что теперь уведомления отправляются через SMS.
2. Добавьте в `AppConfig` создание бина для `PushService` (который вы создали в предыдущей части). Измените `notificationManager()` так, чтобы он использовал новый сервис.
3. (Дополнительное задание) Создайте еще один конфигурационный класс `AnotherConfig` и импортируйте его в главный класс через `@Import({AppConfig.class, AnotherConfig.class})`. Перенесите создание одного из сервисов в новый конфигурационный класс. Убедитесь, что приложение продолжает работать.

Часть 4. Аннотации (@ComponentScan, @Service, @Autowired)

Задача: Перейти на автоматическое сканирование компонентов и избавиться от явного объявления бинов в Java Config.

1. Добавьте аннотации @Service на классы сервисов:

```
1 package org.example.service;
2
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class EmailService implements MessageService {
7     @Override
8     public void sendMessage(String message, String recipient) {
9         System.out.println("EMAIL to " + recipient + ": " +
10             message);
11     }
12 }
```

```
1 package org.example.service;
2
3 import org.springframework.stereotype.Service;
4
5 @Service
6 public class SmsService implements MessageService {
7     @Override
8     public void sendMessage(String message, String recipient) {
9         System.out.println("SMS to " + recipient + ": " +
10             message);
11     }
12 }
```

2. Добавьте @Service на NotificationManager и используйте внедрение через конструктор:

```
1 package org.example.service;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.stereotype.Service;
5
6 @Service
7 public class NotificationManager {
8     private final MessageService messageService;
9
10     @Autowired // Можно опустить, если конструктор один
11     public NotificationManager(MessageService messageService) {
12         this.messageService = messageService;
13     }
14
15     public void notify(String message, String recipient) {
16         messageService.sendMessage(message, recipient);
17     }
18 }
```

```
18 }
19
```

3. Упростите контроллер — теперь можно внедрять NotificationManager напрямую:

```
1 package org.example.controller;
2
3 import org.example.service.NotificationManager;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RequestParam;
6 import org.springframework.web.bind.annotation.RestController;
7
8 @RestController
9 public class NotificationController {
10
11     private final NotificationManager notificationManager;
12
13     public NotificationController(NotificationManager
notificationManager) {
14         this.notificationManager = notificationManager;
15     }
16
17     @GetMapping("/notify")
18     public String notify(@RequestParam String message,
@RequestParam String email) {
19         notificationManager.notify(message, email);
20         return "Уведомление отправлено (аннотации)";
21     }
22 }
23
```

4. Измените AppConfig: удалите все @Bean методы, оставьте только @ComponentScan:

```
1 package org.example.config;
2
3 import org.springframework.context.annotation.ComponentScan;
4 import org.springframework.context.annotation.Configuration;
5
6 @Configuration
7 @ComponentScan("org.example") // Сканируем все компоненты в пак
    ete org.example
8 public class AppConfig {
9     // Никаких @Bean методов!
10 }
11
```

5. Убедитесь, что главный класс по-прежнему импортирует AppConfig:

```
1 @SpringBootApplication
2 @Import(AppConfig.class)
3 public class Application { ... }
4
```

6. Запустите приложение.

Что произошло? Spring нашел два бина типа `MessageService` (`emailService` и `smsService`) и не знает, какой выбрать для внедрения в конструктор `NotificationManager`. Возникает ошибка:

```
1 No qualifying bean of type 'MessageService' available: expected
  single matching bean but found 2: emailService,smsService
```

Получена классическая проблема неоднозначности бинов.

Самостоятельное задание к Части 4

1. Проанализируйте почему возникла ошибка.
2. Временно удалите аннотацию `@Service` с одного из классов-сервисов. Запустите приложение снова. Что изменилось? Почему?
3. Добавьте в `AppConfig` аннотацию `@ComponentScan` с другим пакетом (например, `"org.example.controller"`). Что произойдет с бинами из пакета `service`? Почему?
4. (*Дополнительное задание*) Создайте интерфейс `AdvancedMessageService`, который расширяет `MessageService` и добавляет метод `String getServiceType()`. Реализуйте его в одном из сервисов. Как это повлияет на процесс внедрения зависимости?

Часть 5. Разрешение конфликта бинов

Задача: Научиться управлять выбором бина, когда их несколько.

Способ 1. `@Primary` (основной бин)

Добавьте `@Primary` над одним из сервисов:

```
1 @Service
2 @Primary
3 public class EmailService implements MessageService {
4     // ...
5 }
```

Запустите — ошибка исчезла. Spring будет использовать `EmailService` по умолчанию.

Способ 2. `@Qualifier` (явное указание)

Уберите `@Primary` и добавьте `@Qualifier` в конструктор `NotificationManager`:

```
1 @Service
2 public class NotificationManager {
3     private final MessageService messageService;
4
5     @Autowired
6     public NotificationManager(@Qualifier("smsService")
7     MessageService messageService) {
8         this.messageService = messageService;
9     }
10    // ...
11 }
```

Запустите — теперь используется `SmsService`.

Способ 3. Внедрение коллекции (самый гибкий)

Измените `NotificationManager` так, чтобы он работа со **всеми** доступными сервисами:

```
1 @Service
2 public class NotificationManager {
3     private final List<MessageService> messageServices;
4
5     @Autowired
6     public NotificationManager(List<MessageService> messageServices)
7     {
8         this.messageServices = messageServices;
9     }
10
11     public void notify(String message, String recipient) {
12         messageServices.forEach(service ->
13             service.sendMessage(message, recipient));
14     }
15 }
```

Запустите — теперь уведомление отправляется **и по email, и по SMS** одновременно.

Самостоятельное задание к Части 5

1. **@Primary:** Сделайте так, чтобы основным стал `SmsService`, используя только `@Primary` (не удаляя и не изменяя другие аннотации).
2. **@Qualifier:** Измените имя бина для `EmailService` через `@Service("customEmail")`. Исправьте `@Qualifier` в конструкторе, чтобы он использовал новое имя. Что произойдет, если в `@Qualifier` указать несуществующее имя?
3. **Коллекция:** Добавьте четвертый сервис (например, `TelegramService`) и убедитесь, что он автоматически попадает в список без изменения кода `NotificationManager`. Почему это работает?
4. **Комбинирование:** Попробуйте одновременно использовать `@Primary` на одном сервисе и внедрение `List<MessageService>` в другом месте. Как это работает?
5. (*Дополнительное задание*) Внедрите `Map<String, MessageService>` вместо `List<MessageService>`. Измените метод `notify` так, чтобы он отправлял уведомления только определенными типами сервисов на основе переданного параметра.

Контрольные вопросы

1. В чем разница между `@Component`, `@Service`, `@Repository` и `@Controller`?
2. Как работает `@ComponentScan`? Что произойдет, если указать неправильный пакет?
3. Почему в Части 3 не возникло ошибки с двумя бинами, а в Части 4 возникла?
4. Какие способы разрешения конфликта бинов вы знаете? Какой самый гибкий?

5. Как внедрить все бины определенного типа в коллекцию или карту?
6. В чем преимущества внедрения через конструктор перед внедрением через поле?